



Security Audit

Pruv Finance 2nd (Tokenization Platform)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Centralisation	24
Conclusion	25
Our Methodology	26
Disclaimers	28
About Hashlock	29

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Pruv Finance team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Pruv Finance is an RWA infrastructure layer bridging real-world assets (RWAs) and DeFi. It enables users to access trusted, tokenized assets that are freely tradable and fully composable across decentralized ecosystems.

Pruv streamlines the full lifecycle of real-world assets—from tokenization to distribution and DeFi integration—within a secure, compliant framework. The platform empowers issuers to tokenize assets such as real estate, commodities, and private equity, launch tokenized offerings, and manage them end-to-end.

For investors, Pruv provides seamless access to verified real-world assets that can be subscribed to, held, and redeemed through a transparent, regulated infrastructure. With built-in KYC/AML processes, encrypted transaction records, on-chain token verification, and robust custody solutions, Pruv ensures both compliance and trust at every step of the journey.

Project Name: Pruv Finance

Project Type: Tokenization Platform

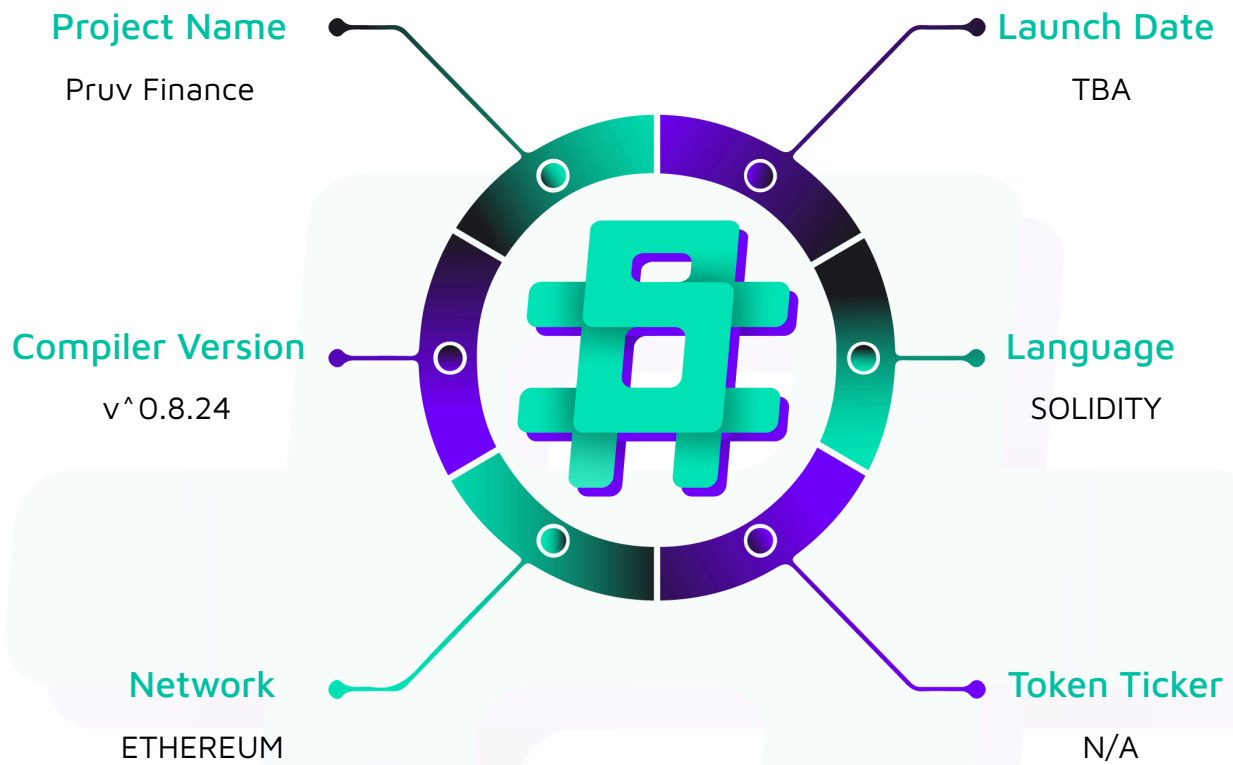
Compiler Version: ^0.8.24

Website: <https://pruv.finance/>

Logo:



Hashlock Pty Ltd

Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Pruv Finance project. The scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Pruv Finance Smart Contracts
Platform	Ethereum / Solidity
Audit Date	October, 2025
Contract 1	SenderWhitelistHook.sol
Contract 2	RouterFeeCollector.sol
Contract 3	HypFiatTokenWithFee.sol
Contract 4	HypERC20CollateralWithFee.sol
Audited GitHub Commit Hash	82fc26c3e342024db74263edc9785127a36a599c
Fix Review GitHub Commit Hash	24ae62c81433514aab4a0d70b1a53c69be0dfb9f

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

2 Medium severity vulnerabilities

3 Low severity vulnerabilities

1 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
SenderWhitelistHook.sol A Hyperlane post-dispatch hook that restricts cross-chain message sending only to whitelisted addresses. Allows the owner to: - Add and remove addresses from the whitelist	Contract achieves this functionality.
RouterFeeCollector.sol An ownable contract for collecting and managing ERC20 token fees from cross-chain transfers. Allows the owner to: - Add and remove fees for specific destination chains - Change the address that receives collected fees - Enable/disable the fee collector - Withdraw all collected fees to the beneficiary Allows the beneficiary to: - Withdraw all collected fees to their address	Contract achieves this functionality.
HypFiatTokenWithFee.sol A Hyperlane-enabled fiat token bridge that burns tokens on the source chain and mints on the destination, with fee collection. Allows the owner to: - Initialize contract with hook, security module, and fee collector Allows the user to: - Transfer fiat tokens to other chains, paying the transfer fee in the same fiat token to the fee	Contract achieves this functionality.

collector	
HypERC20CollateralWithFee.sol A Hyperlane-enabled ERC20 token bridge that locks tokens on the source chain and unlocks them on the destination, with fee collection. Allows the owner to: - Initialize contract with hook, security module, and fee collector Allows the user to: - Send ERC20 tokens to other chains, paying the transfer fee in the fee collector's configured token	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Pruv Finance project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Pruv Finance project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] HypFiatTokenWithFee#transferRemote - Inconsistent amount sent due to fee deduction

Description

The `transferRemote()` method subtracts the fee from `_amountOrId` before sending, so the recipient receives less than the amount specified. This conflicts with the interface expectation `ITokenBridge.transferRemote()`, which documents `_amount` as the amount to send to the recipient.

Vulnerability Details

`ITokenBridge.transferRemote()` expects `_amount` to be the amount delivered to the recipient. However, `HypFiatTokenWithFee` reduces the amount by the fee before dispatch:

```
// Interface: ITokenBridge.sol

/**
 * @notice Transfer value to another domain
 *
 * @param _destination The destination domain of the message
 * @param _recipient The message recipient address on `destination`
 * @param _amount The amount to send to the recipient
 * @return messageId The identifier of the dispatched message.
 */

function transferRemote(
    uint32 _destination,
    bytes32 _recipient,
```

```

    uint256 _amount
) external payable returns (bytes32);
// Contract: HypFiatTokenWithFee.sol

function transferRemote(
    uint32 _destination,
    bytes32 _recipient,
    uint256 _amountOrId
) external payable override nonReentrant returns (bytes32 messageId) {
    uint256 transferFee = feeCollector.quoteFee(_destination);
    require(_amountOrId > transferFee, "Transfer amount must be greater than fee");
    // Collect fee first (Checks-Effects-Interactions pattern) - only if fee > 0
    if (transferFee > 0) {
        IERC20(wrappedToken).safeTransferFrom(msg.sender, address(feeCollector),
transferFee);
    }
    @> uint256 amountAfterFee = _amountOrId - transferFee;
    @> return _transferRemote(_destination, _recipient, amountAfterFee, msg.value);
}

```

By contrast, `HypERC20CollateralWithFee` charges the fee but sends the full `_amountOrId`.

This inconsistency can break integrators that rely on the interface's semantic guarantee that `_amount` is the delivered amount.

Impact

Users and integrators specifying an amount will deliver less than expected on `HypFiatTokenWithFee`, causing underpayments, failed business logic, or accounting discrepancies across domains.

Recommendation

Consider charging the fee on top of `_amount0rId` so that the recipient receives the full specified amount. Require the sender to have approved/supplied the `amount + fee` and transfer the fee separately to the collector while forwarding `_amount0rId` unchanged. This keeps behavior consistent across all token implementations and the interface.

Status

Resolved

[M-02] HypERC20CollateralWithFee - quoteTransferRemote not overridden to include fee approval

Description

HypERC20CollateralWithFee charges a fee during transferRemote(), but it does not override quoteTransferRemote() to include this additional approval requirement. Integrators relying on quoteTransferRemote() for approvals will miss approving the fee token amount, leading to failed transfers.

Vulnerability Details

HypERC20CollateralWithFee.transferRemote() collects a fee in the fee token before dispatch, requiring prior approval for the fee token.

The base contract HypERC20Collateral.quoteTransferRemote() returns quotes for gas payment and the token amount, but does not include the fee added in HypERC20CollateralWithFee.

```
// Contract: HypERC20Collateral.sol

function quoteTransferRemote(
    uint32 _destinationDomain,
    bytes32 _recipient,
    uint256 _amount
) external view virtual override returns (Quote[] memory quotes) {
    quotes = new Quote[](2);
    quotes[0] = Quote({
        token: address(0),
        amount: _quoteGasPayment(_destinationDomain, _recipient, _amount)
    });
    quotes[1] = Quote({token: address(wrappedToken), amount: _amount});
}

// Contract: HypERC20Collateral.sol
```

```

function quoteTransferRemote(
    uint32 _destinationDomain,
    bytes32 _recipient,
    uint256 _amount
) external view virtual override returns (Quote[] memory quotes) {
    quotes = new Quote[](2);
    quotes[0] = Quote({
        token: address(0),
        amount: _quoteGasPayment(_destinationDomain, _recipient, _amount)
    });
    quotes[1] = Quote({token: address(wrappedToken), amount: _amount});
}

// Contract: HypERC20CollateralWithFee.sol

function transferRemote(
    uint32 _destination,
    bytes32 _recipient,
    uint256 _amountOrId
) external payable override nonReentrant returns (bytes32 messageId) {
    uint256 transferFee = feeCollector.quoteFee(_destination);

    // Collect fee first (Checks-Effects-Interactions pattern) - only if fee > 0
    if (transferFee > 0) {
        IERC20(feeCollector.feeTokenAddress()).safeTransferFrom(msg.sender,
address(feeCollector), transferFee);
    }

    return _transferRemote(_destination, _recipient, _amountOrId, msg.value);
}

```

Impact

Callers will not be informed to approve the `transferFee` amount of `feeCollector.feeTokenAddress()`, causing approval underflow and reverted transactions.

Recommendation

Override `quoteTransferRemote()` to include an additional `Quote` for the fee token with `amount = feeCollector.quoteFee(_destinationDomain)`.

In case M-01 is fixed, consider also implementing a similar fix for `HypFiatTokenWithFee`, as the approved amount required would be increased.

Status

Resolved

Low

[L-01] All contracts - Floating pragma

Description

All contracts use floating pragma `pragma solidity >=0.8.0`, and `pragma solidity ^0.8.13` in the case of RouterFeeCollector, instead of fixed version pragmas.

Vulnerability Details

Floating pragmas allow contracts to be compiled with different compiler versions than those tested, potentially introducing unidentified security issues.

Impact

Different pragma versions in test and mainnet may pose unidentified security issues.

Recommendation

Consider locking the pragma version to a specific version, like `pragma solidity 0.8.13`.

Status

Resolved

[L-02] RouterFeeCollector#setFee - no maximum fee cap

Description

The contract allows the owner to set per-destination fees without any upper bound. There is no maximum fee constraint enforced when updating fees.

Vulnerability Details

The `setFee()` function allows the owner to set a new fee value for a destination, without validating that the new fee is below a reasonable maximum.

This enables setting excessively large fees for a destination, which can lead to unexpected charges or make operations economically unviable for users who must pay this fee.

While only the owner can call `setFee()`, the lack of constraints increases the risk of a compromised owner key, misconfiguration, or error.

Impact

Users could face economic denial of service or unexpected excessive fees.

Recommendation

Consider introducing a maximum fee variable enforced in `setFee()` via `require(fee <= maxFee, "fee too high")`.

Status

Resolved

[L-03] All contracts - Centralization risk

Description

Several critical administrative functions are restricted to a single owner. While this is functionally correct, it concentrates control and creates a single point of failure. If the owner key is compromised or misused, privileges, parameters can be changed in ways that disrupt operations.

Vulnerability Details

The contracts implement a single owner address that has control over critical functions. These privileges can affect message routing, access control, fees charged to users, and operational liveness.

Impact

A compromised or malicious owner can cause actions that disrupt operation (e.g., disabling features, altering fees, modifying hooks/ISM), leading to economic DoS or misrouting.

Recommendation

Consider using a multisig wallet for the address controlling the owner role. This would require multiple approvals for critical actions.

Status

Resolved

QA

[Q-01] HypFiatTokenWithFee, HypERC20CollateralWithFee - Missing NatSpec documentation

Description

The contracts `HypFiatTokenWithFee` and `HypERC20CollateralWithFee` lack `NatSpec` documentation that explains their intended functionality. This makes the code harder to understand, maintain, and audit.

Vulnerability Details

Both contracts lack `NatSpec` documentation for contract and function purposes, parameters, and return values.

Impact

Lack of documentation makes the code harder to understand, maintain, and integrate with, while also reducing audit efficiency.

Recommendation

Add comprehensive `NatSpec` documentation.

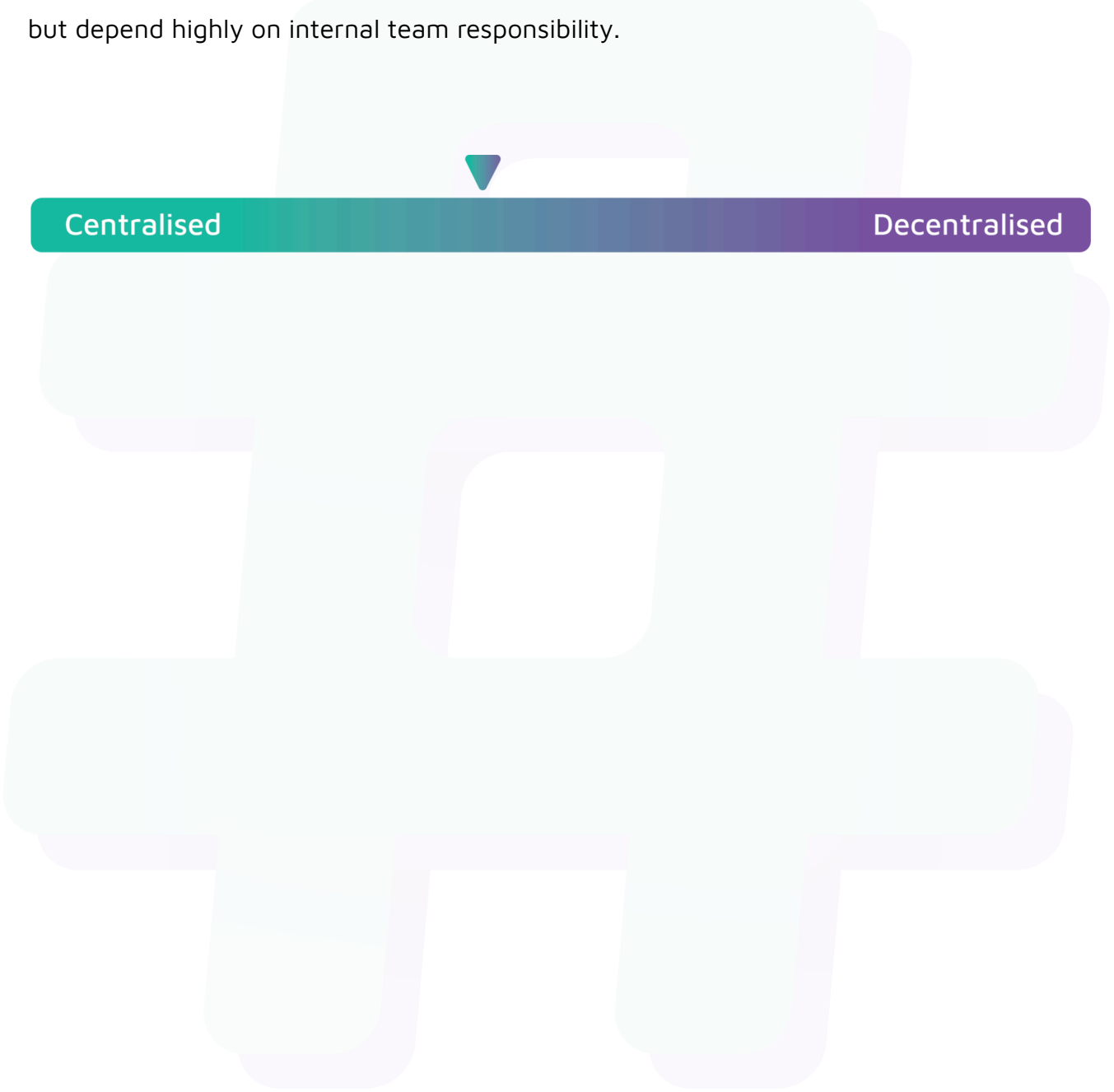
Status

Resolved

Centralisation

The Pruv Finance project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Pruv Finance project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.